

OpenDccCLib

Quick Start Guide — Command Station

Get a DCC command station running in minutes

TI MSPM0G3507 LaunchPad + Code Composer Studio / Theia edition

OpenDccCLib supports any microcontroller with a C compiler — this guide covers one specific path.

Table of Contents

1. What is DCC?
2. What You Need
3. Project Setup for Code Composer Studio
4. Understanding `dcc_user_config.h`
5. Building and Flashing
6. Using the UART Command Interface
7. What's Next

1. What is DCC?

This guide uses a TI MSPM0G3507 LaunchPad with Code Composer Studio / Theia as a concrete, working example. OpenDccCLib itself is platform-agnostic C — it can be adapted to any microcontroller with a C compiler and a timer peripheral. See the Developer Guide for other platforms and IDEs.

DCC (Digital Command Control) is the NMRA standard for controlling model railroad locomotives and accessories. A command station generates the DCC signal — a modulated square wave on the track rails that carries addressed speed, function, and programming commands to decoders installed in locomotives and accessories.

OpenDccCLib is a portable C library that implements the DCC protocol for microcontrollers, handling all low-level bit encoding, packet scheduling, service mode programming, and RailCom detection so you can focus on what your command station actually does.

Key Terms

- **Command Station:** The device that generates DCC packets on the track.
- **Decoder:** A device inside a locomotive or accessory that receives and acts on DCC commands.
- **Packet:** A sequence of bytes encoding one command (address + instruction + error check).
- **Preamble:** At least 14 one-bits before each packet for synchronization.
- **Service Mode:** A special programming track mode for reading/writing decoder Configuration Variables (CVs).
- **RailCom:** A bi-directional communication channel where decoders respond during a cutout in the DCC signal.
- **Scheduler:** The library component that manages concurrent packets, priorities, and auto-refresh.

2. What You Need

Hardware

- TI MSPM0G3507 LaunchPad development board (LP-MSPM0G3507)
- H-bridge motor driver board (e.g. L298N or IBT-2) for track power
- DCC-compatible locomotive with decoder, OR a second LaunchPad running the decoder demo
- USB cable for programming and UART terminal
- 12–18V DC power supply for the H-bridge

DCC Signal Wiring

LaunchPad Pin	Function
PA15	Main track DCC signal output (connect to H-bridge IN1/IN2)
PB4	Service track DCC signal output
PB12	Service track current sense input (ACK detection)
PA18	Debug GPIO (scope trigger on packet sent)

Software

- TI Code Composer Studio (CCS) or CCS Theia IDE
- TI SysConfig tool (included with CCS)
- A serial terminal (PuTTY, Tera Term, or CCS Terminal) at 115200 baud
- The OpenDccCLib source code

3. Project Setup for Code Composer Studio

Starting From the Example

The library includes a ready-to-run command station example at:

```
OpenDccCLib/applications/ti_theia/mspm03507_launchpad/command_station/
```

1. Open CCS/Theia and import the project (File > Import > CCS Projects).
2. Point it at the command_station folder.
3. The project references the DCC library source via a relative path to src/dcc/.

Complete Project File Structure

```
command_station/
  command_station.c           <- Main entry point + ISR handlers
  dcc_user_config.h           <- Feature flags & buffer sizes
  uart_command_parser.c/h     <- UART command-line interface
  application_callbacks/
    callbacks_dcc.c/h         <- Your application logic
  application_drivers/
    ti_driverlib_dcc_driver.c/h <- Hardware abstraction (timers, GPIO)
    ti_driverlib_uart_driver.c/h <- UART I/O
  dcc_lib/                    <- Library (do not edit)
    dcc_config.h/c            <- Config struct & lifecycle API
    dcc_bit_encoder.h/c       <- ISR-level bit serialization
    dcc_scheduler.h/c         <- Packet scheduling
    dcc_packet_encoder.h/c     <- Packet construction
    dcc_railcom_cutout.h/c     <- RailCom cutout timing
    dcc_railcom_decoder.h/c    <- RailCom datagram decoding
    dcc_types.h               <- All data types
    dcc_defines.h             <- NMRA protocol constants
  Debug/
    ti_msp_dl_config.c/h      <- SysConfig-generated init
```

4. Understanding dcc_user_config.h

This is the most important file in your project. It tells the library which role to compile and how much memory to allocate. Every value is mandatory for the enabled role.

```
// --- Role Selection ---
#define DCC_COMPILE_COMMAND_STATION    // Packet scheduler, bit encoder, service mode

// --- Service Mode Methods ---
#define DCC_COMPILE_SERVICE_MODE_DIRECT    // Direct CV read/write
#define DCC_COMPILE_SERVICE_MODE_PAGED    // Paged mode
// #define DCC_COMPILE_SERVICE_MODE_REGISTER // Register mode
// #define DCC_COMPILE_SERVICE_MODE_ADDRESS // Address-only mode

// --- Buffer & Pool Sizes ---
#define USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT 24 // Max concurrent packets
#define USER_DEFINED_DCC_MAX_LOCOS 10 // Auto-refresh locomotives
#define USER_DEFINED_DCC_RAILCOM_BUFFER_DEPTH 4 // RailCom receive ring buffer
#define USER_DEFINED_DCC_SERVICE_MODE_RETRIES 3 // Programming retry count
#define USER_DEFINED_DCC_ACK_THRESHOLD_MA 60 // ACK detection threshold (mA)
#define USER_DEFINED_DCC_ACK_MIN_DURATION_US 5000 // ACK min pulse (us)
#define USER_DEFINED_DCC_ACK_MAX_DURATION_US 7000 // ACK max pulse (us)
```

DCC_COMPILE_SERVICE_MODE_ flags require DCC_COMPILE_COMMAND_STATION. The compiler will error if you enable a service mode without the command station role.*

5. Building and Flashing

In Code Composer Studio

1. Open the imported command station project.
2. Build the project (Project > Build or Ctrl+B).
3. Connect the LaunchPad via USB.
4. Flash and run (Run > Debug or F11).
5. Open a serial terminal at 115200 baud on the LaunchPad's COM port.
6. You should see the startup banner: "DCC Command Station - MSPM0G3507 LaunchPad"

What Happens at Power-On

The `main()` function initializes hardware drivers, passes the `dcc_config_t` struct to `DccConfig_initialize()`, then enters the main loop calling `DccConfig_run()` and the UART parser. Track power is OFF by default — type POWER ON to start generating DCC packets.

The Main Loop

```
while (1) {  
    DccConfig_run();           // Library packet scheduling & callbacks  
    TI_UartDriver_echo_process(); // Echo typed characters  
    UartCommandParser_process(); // Parse and execute commands  
}
```

6. Using the UART Command Interface

Type HELP at the terminal to see all commands. Here are the most common:

Command	Description
POWER ON OFF	Enable/disable track power
SPEED <addr> <speed> <FWD REV> [128]	Set locomotive speed (0–126, 128-step mode)
ESTOP [addr]	Emergency stop (all or specific loco)
FUNC <addr> <0-68> <ON OFF>	Toggle function output (lights, sound, etc.)
ACC <board> <pair> <ON OFF>	Throw/close basic accessory (turnout)
ACCE <addr> <aspect>	Set extended accessory signal aspect
SVC ENTER	Enter service mode (programming track)
SVC EXIT	Exit service mode
SVC DIRECT WRITE <cv> <value>	Write CV in direct mode
SVC DIRECT VERIFY <cv> <value>	Verify CV in direct mode
STATUS	Show scheduler and track status
HELP	List all commands

Quick Test

```
> POWER ON
OK: track power ON
> SPEED 3 50 FWD
OK: SPEED addr=3 speed=50 dir=FWD mode=128
> FUNC 3 0 ON
OK: FUNC addr=3 f0=ON
> ESTOP
OK: ESTOP broadcast
```


7. What's Next

Add RailCom Detection

Wire a RailCom detector's UART output to a 250 kbaud UART peripheral. Populate the `dcc_railcom_hw_t` struct and point `main_track.railcom` to it. The library handles cutout timing, datagram decoding, and fires `on_railcom_datagram_result`.

Add Current-Sense Protection

Wire an ADC or comparator to the main track current sense. Set `main_track.current_sense_read` to your read function. The library uses this for overcurrent protection.

Port to Another MCU

Copy the `command_station` project. Rewrite the two driver files (`ti_driverlib_dcc_driver.c/h`) for your MCU's timer and GPIO API. The library never calls hardware directly — all hardware access goes through the function pointers in `dcc_config_t`.

Read the Developer Guide

The companion OpenDccCLib Command Station Developer Guide walks through every part of the system in detail: the config struct, scheduler internals, bit encoding, RailCom cutout, service mode state machines, and more.